Cheatsheet AZ-104 — 10 · Containers: Container Registry, Container Instances and Container Apps

1. Choosing the service

Service	What it is	When	Not suitable if
Azure Container Instances (ACI)	**Standalone** containers (container groups) without an orchestrator, billed per second	One-off tasks, batch, testing, **a quick container**, AKS bursting (virtual nodes)	You need **autoscale**, load balancing across replicas, revisions or managed HTTP ingress
Azure Container Apps (ACA)	**Serverless** microservices and HTTP / event-driven apps on managed Kubernetes (KEDA, Dapr, Envoy), **scales to zero**	APIs, containerized web apps, queue workers, jobs; no cluster to manage	You need access to the Kubernetes API, Windows containers or node control
AKS	Full Kubernetes	Full control, operators, complex workloads	You want something simple (AZ-104 barely asks about it)
App Service (Web App for Containers)	Web PaaS with your own image	Classic containerized web app with slots, domains, TLS	Complex multi-container, scale to zero
Azure Container Registry (ACR)	**Private** image registry (Docker / OCI)	Whenever you use the above with your own images	—

2. Azure Container Registry

Aspect	Basic	Standard	Premium
Included / maximum storage	10 GiB / 40 TiB	100 GiB / 40 TiB	**500 GiB** / 100 TiB
Webhooks	2	10	500
Geo-replication (one registry, several regions)	—	—	**Yes**
Private Link (private endpoints), IP rules, service endpoints	—	—	**Yes**
Content trust (signed images), **customer-managed keys**, **retention policy** (untagged manifests), dedicated data endpoints, connected registries	—	—	**Yes**
Anonymous pull, artifact cache	—	Yes	Yes
Zone redundancy, tokens and scope maps, Entra RBAC per repository	Yes	Yes	Yes
Performance (r/m, concurrency)	Low	Medium	High

- Name: 5-50 alphanumeric characters, **global**; login server `<name>.azurecr.io`. Changing SKU has no downtime (downgrading from Premium requires removing geo-replicas).
- **Authentication:** Entra ID with the **AcrPull** (pull only), **AcrPush** (push and pull), **AcrDelete**, **AcrImageSigner** roles; **managed identity** for AKS / ACA / App Service / VMs; **service principal** for CI/CD; **admin user** (two passwords; disabled by default; testing only); tokens with scope maps for specific repositories. "Least privilege for AKS to pull" → AcrPull to the cluster identity.
- **ACR Tasks:** `az acr build -r acr1 -t app:v1 .` builds in the cloud **without local Docker**; automatic tasks on Git **commit**, on **base image update** or on a schedule; multi-step tasks (YAML); `az acr run`. **Import** from Docker Hub or another registry: `az acr import -n acr1 --source docker.io/library/nginx:latest -t nginx:latest`.
- Operate: `az acr login -n acr1` → `docker tag app acr1.azurecr.io/app:v1` → `docker push acr1.azurecr.io/app:v1` · `az acr repository list -n acr1` · `az acr repository show-tags -n acr1 --repository app` · `az acr repository delete -n acr1 --image app:v1` · `az acr show-usage`. **Defender for Containers** scans for vulnerabilities. **Image lock** (`az acr repository update --write-enabled false`) prevents overwriting or deleting a tag.

3. Azure Container Instances

Aspect	Detail
Container group	Deployment unit: one or more containers (up to 60, **multi-container Linux only**) sharing lifecycle, network (same IP and ports) and volumes; sidecar pattern; **init containers**
Resources	**CPU and memory** are set per container (`--cpu 1 --memory 1.5`); the group can reach tens of vCPUs, but above 4 vCPU / 16 GB it depends on regional capacity; billed **per second** of vCPU and GB
Images	Linux (x64) and **Windows**; from ACR with `--registry-login-server --registry-username --registry-password` or **managed identity** (`--acr-identity`)
Networking	**Public IP** with DNS label (`--dns-name-label app1` → `app1.<region>.azurecontainer.io`) and ports (`--ports 80 443`, max. 5 per IP) **or** deployed in a **VNet** (`--vnet --subnet`, subnet delegated to `Microsoft.ContainerInstance/containerGroups`, **private** IP; not both)
Storage	**Azure Files** (`--azure-file-volume-account-name/-key --azure-file-volume-share-name --azure-file-volume-mount-path`; Linux only), emptyDir, gitRepo, secret volumes
Restart	`--restart-policy Always` (default; services) · `OnFailure` (tasks) · `Never` (run once)
Configuration	Variables `--environment-variables`, secure ones `--secure-environment-variables`, command `--command-line`; YAML (`az container create -f group.yaml`) for multi-container
Operate	`az container logs -n g1 -g RG1` · `az container attach` (live stdout) · `az container exec -n g1 --exec-command "/bin/sh"` · `az container show --query ipAddress.fqdn` · `az container restart / stop / start / delete`
Scaling	**No autoscale**: create more groups (or use ACA / AKS); **Spot containers** (preview) and **confidential containers**

Aspect	Detail
Quotas	By default 100 container groups and 100 vCPUs per region and subscription (can be raised); no built-in replicas or redundancy (HA = several groups behind your own load balancer)

- Typical case: "run a one-off job / build / batch without managing VMs, billed per second" → ACI with `--restart-policy Never`. "Persistent data across restarts" → Azure Files volume. "Accessible only inside the VNet" → deploy in the delegated subnet.

4. Azure Container Apps

Piece	What it is	Detail
Environment	Shared secure boundary (network, Log Analytics, Dapr) for several apps	**Consumption only** or **workload profiles** (Consumption + **Dedicated** D4/D8/E4... with more CPU and GPU); can live in your **VNet** (internal or external); apps in the same environment talk to each other by name
Container app	Image (Linux x64), resources, secrets, identity, **ingress**, scaling	Several containers per app (sidecar / init) only if they are tightly coupled; `az containerapp up` creates environment + app in one step
Revisions	Every change to the `template` (image, env, resources) creates an immutable **revision**; **single** mode (the new one replaces) or **multiple** (several active)	**Traffic splitting** by % between revisions (blue-green, canary), labels with their own URL; "roll back" = send traffic back to the previous one
Ingress	**External** (public, managed HTTPS, custom domain and certificate), **Internal** (environment / VNet only) or disabled; HTTP, TCP and gRPC; target port; IP restrictions; sticky sessions	"only reachable from other apps" → internal
Scaling	**Min / max replicas** (min 0 = **scale to zero**; max. 1,000 replicas); **HTTP** rules (concurrent requests), **TCP**, **custom** via **KEDA** scalers (Service Bus, Storage queue, Event Hubs, CPU, memory...). CPU / memory **do not scale to zero**	Without rules: 1 fixed replica between min and max; configurable cooldown and polling
Resources	Fixed combinations in Consumption: **0.25 vCPU / 0.5 Gi** ... up to **4 vCPU / 8 Gi** (memory = 2 × CPU); dedicated workload profiles allow more; in a *Consumption only* environment the maximum is 2 vCPU / 4 Gi	"0.5 CPU and 2 GiB" is not valid (it would be 0.5 / 1 Gi)
Jobs	Containers that **finish**: manual, **scheduled** (cron) or **event-triggered** (KEDA)	Replace ACI for tasks inside the environment
Secrets and identity	Own secrets or **Key Vault** references; **managed identity** (system / user) for ACR (`acrPull`) and other services	"no credentials in the app"
Dapr	Service invocation, pub/sub, state stores	Optional
Logs	Console and system in **Log Analytics** (`ContainerAppConsoleLogs_CL`, `ContainerAppSystemLogs_CL`) or Azure Monitor; `az containerapp logs show`	—

```
az containerapp env create -n env1 -g RG1 -l westeurope [--infrastructure-subnet-resource-id <subnet>] [--internal-only true]
az containerapp create -n app1 -g RG1 --environment env1 --image acr1.azurecr.io/app:v1 \
   --registry-server acr1.azurecr.io --registry-identity system --target-port 80 --ingress external \
   --min-replicas 0 --max-replicas 10 --cpu 0.5 --memory 1.0Gi --scale-rule-name http --scale-rule-http-concurrency 50
az containerapp update -n app1 -g RG1 --image acr1.azurecr.io/app:v2            # new revision
az containerapp revision list -n app1 -g RG1 -o table
az containerapp ingress traffic set -n app1 -g RG1 --revision-weight app1--v1=80 app1--v2=20
az containerapp job create -n job1 -g RG1 --environment env1 --trigger-type Schedule \
   --cron-expression "0 2 * * *" --image ... --replica-timeout 1800
```

5. Commands that show up (ACR and ACI)

Task	Azure CLI	PowerShell
Create registry	`az acr create -n acr1 -g RG1 --sku Premium --admin-enabled false`	`New-AzContainerRegistry -ResourceGroupName RG1 -Name acr1 -Sku Premium`
Build and push	`az acr build -r acr1 -t app:v1 .` · `az acr login -n acr1` + `docker push acr1.azurecr.io/app:v1`	`Connect-AzContainerRegistry -Name acr1` + docker push
Import and list	`az acr import -n acr1 --source docker.io/library/nginx:latest -t nginx:latest` · `az acr repository list -n acr1` · `az acr repository show-tags -n acr1 --repository app`	`Get-AzContainerRegistryRepository -RegistryName acr1` · `Get-AzContainerRegistryTag -RegistryName acr1 -RepositoryName app`
Geo-replicate	`az acr replication create -r acr1 -l northeurope`	`New-AzContainerRegistryReplication -RegistryName acr1 -Location northeurope`
Pull permission	`az role assignment create --assignee <mi id> --role AcrPull --scope <acr id>`	`New-AzRoleAssignment -ObjectId <mi> -RoleDefinitionName AcrPull -Scope <acr id>`
Create public ACI	`az container create -g RG1 -n c1 --image acr1.azurecr.io/app:v1 --cpu 1 --memory 1.5 --dns-name-label app1 --ports 80 --os-type Linux --restart-policy Always --acr-identity <id> --assign-identity <id>`	`$p = New-AzContainerInstancePortObject -Port 80 -Protocol TCP; $c = New-AzContainerInstanceObject -Name c1 -Image ... -RequestCpu 1 -RequestMemoryInGb 1.5 -Port $p; New-AzContainerGroup -ResourceGroupName RG1 -Name c1 -Location westeurope -Container $c -OsType Linux -IPAddressType Public -IPAddressDnsNameLabel app1 -RestartPolicy Always`
ACI in a VNet	`az container create ... --vnet vnet1 --subnet sub-aci` (without `--dns-name-label`)	`-SubnetId <id>`
ACI with Azure Files	`az container create ... --azure-file-volume-account-name st1 --azure-file-volume-account-key <key> --azure-file-volume-share-name share1 --azure-file-volume-mount-path /data`	`New-AzContainerGroupVolumeObject -Name v1 -AzureFileShareName share1 -AzureFileStorageAccountName st1 -AzureFileStorageAccountKey <key>`

Task	Azure CLI	PowerShell
Operate ACI	`az container logs / attach / exec / show / restart / delete -g RG1 -n c1`	`Get-AzContainerInstanceLog` · `Get-AzContainerGroup` · `Restart-AzContainerGroup` · `Remove-AzContainerGroup`

6. Decision tree

```
What does the wording ask for?
├─ Store your own images privately ..................................... Azure Container Registry
├─ Registry in several regions / private endpoint / signed images ..... ACR Premium
├─ Build the image without Docker on the local machine ................. ACR Tasks (az acr build)
├─ Let AKS / ACA / App Service pull images without credentials ......... Managed identity + AcrPull
├─ A quick container, task or batch, per-second billing ................ ACI (restart-policy Never / OnFailure)
├─ Container with a private IP in the VNet ............................. ACI in a delegated subnet (or ACA with VNet)
├─ Several containers sharing lifecycle and localhost .................. ACI container group (Linux) or ACA sidecar
├─ Persist a container's data .......................................... Azure Files volume
├─ HTTP API that scales with load and down to zero ..................... Container Apps (ingress + HTTP rules)
├─ Process messages from a queue, scale by queue length ................ Container Apps with a KEDA scaler
├─ Roll out v2 to 10 % of users and roll back .......................... Container Apps: multiple revisions + traffic split
├─ Scheduled task in a container ....................................... Container Apps Job (schedule)
├─ Only reachable from other apps in the environment ................... Ingress internal
└─ Full Kubernetes with kubectl access ................................. AKS
```

7. Signals → answer

Wording	Answer
"private registry with geo-replication"	ACR Premium
"registry must not be reachable from the internet"	ACR Premium + private endpoint (and public access disabled)
"least privilege for the container app to pull images"	AcrPull to its managed identity
"admin user credentials in the deployment"	Bad practice: use an identity / service principal
"build the image in Azure from the Dockerfile"	`az acr build`
"automatically rebuild when the base image updates"	ACR Task with a base image trigger
"delete untagged manifests automatically"	Retention policy (Premium)
"run a container for a few minutes and bill only for that"	ACI, restart policy Never
"container needs a public DNS name"	ACI with `--dns-name-label`
"container group with Windows and Linux containers"	Impossible: multi-container is Linux only
"ACI must access resources in the VNet only"	ACI in a delegated subnet (no public IP)
"data must survive container restarts"	Azure Files volume
"scale to zero when idle"	Container Apps (min replicas 0)
"scale on HTTP concurrent requests / queue length"	Container Apps scale rule HTTP / custom (KEDA)
"0.5 vCPU with 2 GiB" rejected	Fixed combinations: 0.5 vCPU → 1 Gi
"run two versions and split traffic"	Multiple revision mode + traffic weights
"container app only reachable inside the environment"	Ingress internal
"nightly job in a container inside the environment"	Scheduled Container Apps Job
"ACI needs more replicas automatically"	No autoscale in ACI → ACA

8. Mistakes to avoid

1. Choosing ACI when autoscale, revisions or scale to zero are required: that is Container Apps.
2. Expecting Private Link, geo-replication or content trust in ACR Basic / Standard.
3. Using the ACR admin user in production instead of managed identity + AcrPull.
4. Asking for a public IP and a VNet at the same time in a container group, or multi-container on Windows.
5. Forgetting the Azure Files volume and losing data on restart.
6. Setting CPU / memory outside the allowed combinations in Container Apps (memory = 2 × vCPU).
7. Assuming CPU or memory can scale to zero in Container Apps (only HTTP, TCP and event scalers).
8. Leaving ingress external when the wording asks for internal-only access.